Project Brief: Smart Greenhouse Mobile App

1. Overview

Each team builds a working mobile app that lets a greenhouse owner see its condition in real time and control its equipment remotely. The required core is the same for everyone; *who* the product is for and *how* it solves the problem is each team's own choice.

Parameter	Value
Product name	Chosen by the team (not "Smart Greenhouse": the name is part of the concept)
Product type	Software only: mobile app (Android required, iOS optional) + server +
Team format	2–4 people; roles are assigned and recorded in the README
Timeline	8 weeks from brief to final defense
Hardware	Not used. No Arduino, ESP32, sensors or relays — a software simulator replaces them entirely
Deliverables	source code in Git, demo video, 10-minute defense

Learning goal for students: go through the full development cycle — from a user's problem to a working product with data, an interface and a defended design.

2. Required Features (Core)

A project without all eight modules is not admitted to the defense. How they look and work is up to the team.

#	Module	Minimum requirement	How it is checked
M1	Authentication	Login with username/password or a code; data visible only to its owner	Another account cannot see the greenhouse
M2	Dashboard	At **least** 4 parameters: air temperature, air humidity, soil moisture, light level; time of last update	Values change as the simulator changes
M3	Remote control	At least 3 virtual devices: irrigation, ventilation (window/fan), lighting or heating	A command from the app changes the device state

in the simulator, and readings respond (irrigation → soil moisture rises)

#	Module	Minimum requirement	How it is checked
M4	Operating modes	Manual **and** automatic mode; in automatic, threshold rules (e.g. "soil moisture < 30% → water for 2 min")	The rule fires without user action
M5	Notifications	Push or in-app alerts when a parameter leaves its allowed range	Alert demonstrated at the defense
M6	History and charts	Readings stored for at least 7 days; chart of at least 2 parameters with a selectable period	Chart is built from real stored data
M7	Settings	User sets thresholds, schedule and units	Changed thresholds affect the automation
M8	Failure handling	Clear "no connection to greenhouse" state; the app never crashes	At the defense the data source is cut off and the app behaves correctly

Key user scenarios the team shows in the demo:

1. The user opens the app and within 5 seconds understands whether everything in the greenhouse is fine.
2. They get an overheating alert, open the window remotely and watch the temperature drop.
3. They set up automatic watering and leave for the weekend.
4. They look at the weekly chart and find when the plants lacked water.

3. Technical Requirements

The system is entirely software and has three parts: a greenhouse simulator, a server with a database, and the mobile app. The team chooses the stack for each part and justifies the choice at the defense.

Part	Allowed options	Required
Mobile app	Kotlin, Java, Flutter, React Native	At least 5 screens; adapts to different screen sizes
Server and data	Firebase, Supabase, custom backend (Node.js, Python, PHP), MQTT broker	Stores history; the app reads data and sends commands only through the server

Greenhouse simulator	A script or service in Python, Node.js, Java or any language; runs on a PC or in the cloud	Sends readings to the server at least once every 30 seconds and accepts control commands

Non-functional requirements

• A control command reaches the device in no more than 5 seconds on a normal connection. • App cold start takes no more than 3 seconds on an average smartphone.
• Passwords and API keys are never stored in plain text in the code or the repository. • Interface in Russian or Kazakh (a second language earns credit in the creative part). • Code in Git: at least 20 meaningful commits from each member, plus a README with run instructions.

Architecture diagram — draw your own and submit it with the project (see section 5).

4. Creative Zone: Making the Project Your Own

Two projects with the same core must not look or work the same. To ensure this, every team follows three freedom rules and picks at least one uniqueness track.

Three freedom rules (required for everyone)

1. **Your own persona.** A specific user with a name, age, situation and pain point. No two teams in a group may take the same persona — topics are assigned in week 1.
2. **Your own signature feature.** One feature that is not in this brief and solves a real problem for this particular persona. The team states it in one sentence: "Our app can ___ because our user ___."
3. **Your own visual identity.** Original name, logo, color scheme and style. Tutorial template designs without substantial rework are not accepted.

Uniqueness tracks (choose 1–2)

Track	Idea	Example directions
Agro-intelligence	The app helps make decisions, not just display numbers	Crop profiles (tomatoes, cucumbers, strawberries) with preset thresholds; forecasts like "overheating in 2 hours"; care tips
Resource savings	Measure and reduce water and electricity use	Irrigation liter counter; "how many tenge the automation saved" report; watering that accounts for the weather forecast
Scale	Multiple greenhouses and	Greenhouse list; "owner / worker" roles; log of who switched what and when

	users	
Accessibility	A product for people who struggle with technology	Large interface for older users; voice commands; Kazakh language; "one big status" mode
Education	The greenhouse as a learning tool	Plant growth diary with photos; tasks for schoolchildren; comparing conditions in two beds

Track	Idea	Example directions
Your own track	The team proposes its own direction	Must be agreed with the instructor by the end of week 2

Not allowed: copying another team's project or a ready-made solution from the internet without substantial rework; replacing functionality with attractive but non-working screens. At the defense, each member explains any part of the code in their area of responsibility.

5. Stages, Deliverables and Defense

The project runs for 8 weeks with three checkpoints; a missed checkpoint lowers the final score (see the rubric, criterion 6).



project stages · 5 phases, 3 checkpoints

At CP1 the team submits the project charter, at CP2 it shows working M1–M3, and at CP3 it defends the finished product.

What to submit

Deliverable	Due	Contents

Project charter	CP1	Name, persona (1 page), chosen track, signature feature statement, team roles
Mockups and diagram	Week 3	Mockups of all screens (Figma or similar), architecture diagram with data flows
Repository	Throughout	Code, README with run instructions, commit history of all members
APK	CP3	Installable file, test account for review
Demo video	CP3	2–3 minutes: the four key scenarios from section 2

Deliverable	Due	Contents
Presentation	CP3	Up to 8 slides: persona's problem → solution → demo → next steps

Defense format (10 minutes): 5 minutes — live demo with simulator data; 2 minutes — a "stress test" by the panel (stopping the simulator, triggering a failure scenario, entering invalid thresholds); 3 minutes — questions to each member about their part of the code.

6. Grading Rubric (100 points)

Projects are graded on six criteria: 70% of the points go to what works and how well it is built, 30% to the team's own decisions and its ability to defend them.

#	Criterion	Max	Excellent	Good	Satisfactory	Insufficient
1	Required features M1–M8	30	**27–30.** All 8 modules work on live data, automation fires, stress test passed with no failures	**20–26.** All modules present, 1–2 work partially or with delays	**12–19.** 5–6 modules work, the rest are stubs	**0–11.** Modules exist on paper but do not work with data

2	Creativity and uniqueness	20	**18–20.** Persona is well developed; the signature feature solves its real problem and works; track implemented in depth; product is recognizable	**13–17.** Feature works but is loosely tied to the persona, or the track is partly developed	**7–12.** Feature is declared but implemen te d superficial ly; design close to a template	**0–6.** No own feature, or the project repeats someone else's / a tutorial
3	UX/UI	15	**14–15.** Greenhouse status is clear within 5 seconds; consistent style; interface tailored to the persona	**10–13.** Convenient, but with extra steps or style inconsisten cie s	**5–9.** Usable only with guidance	**0–4.** Confusing navigation, data hard to read
4	Technical quality and architect ure	15	**14–15.** Clean code structure, architecture diagram matches the implementation, no keys in code, README lets anyone run the project from scratch	**10–13.** Working code with minor structure or security issues	**5–9.** Code hard to read, diagram does not match the	**0–4.** Passwords in the repository, project does not run from the README

#	Criterion	Max	Excellent	Good	Satisfactory	Insufficient
					impleme nta tion	
5	Defense and understan di ng	10	**9–10.** Every member confidently explains their part of the code and justifies the stack	**6–8.** Answers most questions, with some uncertainty	**3–5.** Only part of the team understa nds the	**0–2.** Cannot explain their own code

			choice		code	
6	Process and teamwork	10	**9–10.** All 3 checkpoints on time, steady commits from every member	**6–8.** One checkpoint late or uneven contribution	**3–5.** Two checkpoints late, work done in the last week	**0–2.** Checkpoints missed, code written by one person

Converting to a grade

Points	Grade
90–100	5 (excellent)
75–89	4 (good)
60–74	3 (satisfactory)
0–59	2 (fail, rework required)

Grading rules

- **Admission to defense:** all modules M1–M8 are present in the app. Otherwise the project is returned for rework.
- **Individual score:** the team total is adjusted for each member by ±10 points based on criteria 5 and 6 (defense answers and commit history).
- **Anti-copy rule:** if two projects match in persona, signature feature and design, both teams receive 0 for criterion 2 until authorship is established.
- **Boldness bonus (up to +5):** for an ambitious but well-justified idea that works at least partially. The total cannot exceed 100 points.